

Digital Signature Using RSA

Practical / Laboratory Program

1. Aim

To implement a basic RSA digital signature scheme by generating a key pair, signing a message using the private key, and verifying the signature using the public key.

2. Algorithm

1. Start the program.
2. Choose two prime numbers p and q .
3. Calculate $n = p \times q$ and $\phi(n) = (p - 1)(q - 1)$.
4. Choose a public exponent e that is relatively prime to $\phi(n)$.
5. Calculate the private exponent d such that $(d \times e) \bmod \phi(n) = 1$.
6. Convert the message to a numeric value m smaller than n .
7. Generate the signature $s = m^d \bmod n$.
8. Verify the signature by calculating $m' = s^e \bmod n$.
9. If m' equals m , accept the signature; otherwise reject it.
10. Stop the program.

3. Program

```
# Basic RSA digital signature demonstration
# Educational example only; real systems must use a vetted cryptographic library.

p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = pow(e, -1, phi)

message = input("Enter a numeric message (0-999): ")
m = int(message)

if not (0 <= m < n):
    raise ValueError("Message must be between 0 and 3232.")

signature = pow(m, d, n)
verified_message = pow(signature, e, n)

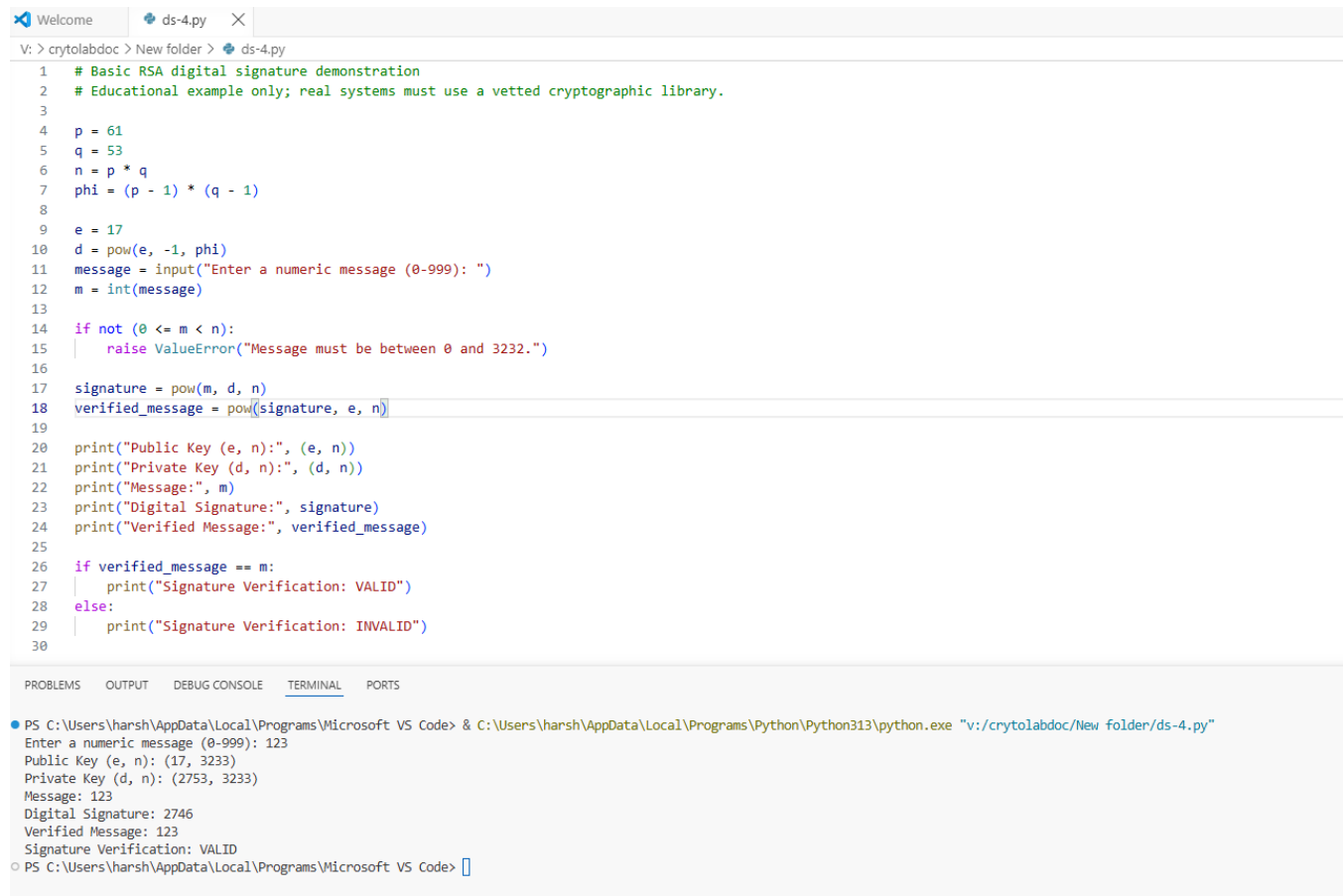
print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
print("Message:", m)
print("Digital Signature:", signature)
print("Verified Message:", verified_message)

if verified_message == m:
    print("Signature Verification: VALID")
else:
    print("Signature Verification: INVALID")
```

4. Input

123

5. Output



```
1 # Basic RSA digital signature demonstration
2 # Educational example only; real systems must use a vetted cryptographic library.
3
4 p = 61
5 q = 53
6 n = p * q
7 phi = (p - 1) * (q - 1)
8
9 e = 17
10 d = pow(e, -1, phi)
11 message = input("Enter a numeric message (0-999): ")
12 m = int(message)
13
14 if not (0 <= m < n):
15     raise ValueError("Message must be between 0 and 3232.")
16
17 signature = pow(m, d, n)
18 verified_message = pow(signature, e, n)
19
20 print("Public Key (e, n):", (e, n))
21 print("Private Key (d, n):", (d, n))
22 print("Message:", m)
23 print("Digital Signature:", signature)
24 print("Verified Message:", verified_message)
25
26 if verified_message == m:
27     print("Signature Verification: VALID")
28 else:
29     print("Signature Verification: INVALID")
30
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\harsh\AppData\Local\Programs\Microsoft VS Code> & C:\Users\harsh\AppData\Local\Programs\Python\Python313\python.exe "v:/cryptolabdoc/New folder/ds-4.py"
Enter a numeric message (0-999): 123
Public Key (e, n): (17, 3233)
Private Key (d, n): (2753, 3233)
Message: 123
Digital Signature: 2746
Verified Message: 123
Signature Verification: VALID
PS C:\Users\harsh\AppData\Local\Programs\Microsoft VS Code>
```

6. Result

Thus, the RSA digital signature was generated successfully and verified successfully. The demonstration shows that a signature created with the private key can be validated with the corresponding public key.